

Wrocław University of Science and Technology

Artificial Intelligence and Computer Vision Project Report:

- Shoe Model Detector

Student: Paweł Ćwikowski

Group no:2

Index no: 278150

Submission Date: 23.01.2026

Lab assistant: dr inż. Cholewiński Mateusz

Grade:

Table of Contents:

1. Executive Summary	4
2. Project Scope	4
3. Case Studies and Research	4
3.1. Existing Solutions	4
3.2. Technical Components	5
3.2.1. OpenCV & Camera UI	5
3.2.2. YOLOvX Models	5
3.2.3. CLIP (Contrastive Language-Image Pre-training)	5
3.2.4. FAISS (Facebook AI Similarity Search)	5
3.2.5. Pricing & Web Scraping APIs	5
4. Workflow	5
4.1. The ID-PEMO Process	5
5. Implementation & Milestones	6
Milestone 1: Gathering the Dataset	6
Milestone 2: Labeling of Dataset	6
Milestone 3: CLIP Integration & Vectorization	7
Milestone 4: YOLO Implementation	8
- Model Selection	8
- Training Parameters & Epochs	9
- Training Results & F1 Scores	10
- Post-training ROI Cropping	11
Milestone 5: Adding FAISS	11
- Phase 1: Indexing	11
- Phase 2: Querying	12

Milestone 6: User Interface	13
- Confidence Scores & Indicators	14
- Status Indicators	14
Milestone 7: Pricing & Web Scraping	15
- DuckDuckGo API Attempts	15
- Mock Database Implementation	15
6. Challenges Encountered	16
6.1. Data Gathering & Labeling Constraints	16
6.2. YOLO Model Training Interruptions	16
6.3. Real-time Latency Optimization	17
6.4. Visual Similarities	17
6.5. API Restrictions	17
7. Conclusions	18
8. Link to GitHub	19

1. Executive Summary

My shoe model detector is an application using computer vision and external detection model for real – time sensing and examining specific objects. The goal is to achieve such output that a model will distinguish geometrical shapes and match unique sneaker patterns and output its model name with their prices. This documentation contains my implementation of YOLOv8 for detection and CLIP for feature extraction models and in the end mock prices of the detected models.

2. Project Scope

From the beginning I had three main goals to achieve:

- a) **Real-time Detection:** So, identifying the presence of a shoe by computer camera using open cv library
- b) **Model Classification:** Distinguishing between specific shoe models (like Adidas Ozelia, New Balance 550, Nike Blazer Mid 77, Nike Dunk Low, Nike Pogo Plus)
- c) **Accuracy of the output Data:** I did not use many various models. Instead, focus for only a few to achieve more reliable final results.

3. Case Studies and Research

3.1. Existing Solutions

The project from this field is rather niche one when it comes to detection and some models there are plenty of them but with combination of shoe, sneaker topic the research I made concluded in a rather small amount of science papers.

What I found there is one about detecting counterfeit model from authentic one using YOLOv8: <https://irojournals.com/iroiip/article/pdf/6/3/8>

and

CNN and AI implementation for shoe recognition by provided photo from the user: <https://www.mdpi.com/2673-4591/92/1/62>

Apart from that I focused to study all parts of the factors in the project that I would need by myself which included:

OpenCV documentation and YouTube videos about camera UI and adjusting factors like exposure frame rate and resolution to in the final obtain precise, steady quality window view

3.2. Technical Components

YOLOvX models – principle of their operations, how they differ from other models existing/ available for common usage, what is the difference between different models of YOLO itself and how this would influence my project.

CLIP (Contrastive Language-Image Pre-training) – The model that is alternative for CNN (neural networks for machine learning) that understand the concept of distinguishing the specific symbols, colors, patterns on shoes itself and match them with the database from models available from my library.

FAISS (Facebook AI Similarity Search) – The innovative mechanism that Facebook introduced on the market. In overall it is used to for fast similarity search and clustering of high-dimensional vectors, crucial for AI tasks like image/product recommendations, finding similar photos you may think of it like Facebook's own systems and even google that advertises that pop up are similar to things you talked, searched in the internet just few minutes ago. In my project I used it to improve the CLIP itself to enable faster model matching from database for CLIP not to think too slow.

Pricing & Web Scraping – I tried to use this tool to help find model prices with easy using a DuckDuckGo Search API. What it does? It searches the internet with the purpose of finding instant answer/ automate search queries on quotes topics questions user put into the code. In my case, it was the question involving prices of specific models of the shoes, for example *"Nike Dunk Low Purple Court price StockX"*) and parse it for the instant answer.

4. Workflow

4.1. The ID-PEMO Process

I made a short description for you to understand what is principle of operation of the program in short ID-PEMO process.

- a) **Input:** The webcam captures a live video stream.
- b) **Detection:** The detection model identifies the bounding box of the shoe.
- c) **Preprocessing:** The image within the bounding box is cropped and normalized.
- d) **Embedding:** The classification model converts the image into a vector.
- e) **Matching:** The vector is compared against the pre-indexed database of 6 shoe models.
- f) **Output:** The system overlays the Shoe Name, Confidence Score, and Price (still in the works) on the screen.

5. Implementation & Milestones

Before I proceed, I would like to show how repository & directory structure looked like at the beginning of the project and then show in comparison to the final file tree in the end of documentation:

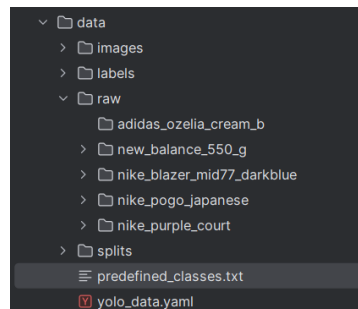


Figure 1. Beginning File Structure

Milestone 1: Gathering the Dataset (Completed: 14.10.2025)

First to complete was the first milestone which mostly consist of gathering necessary amount of photos, images of chosen shoes to prepare the model for pre training in my case the Dataset consisted of overall collected images 280 distinct shoe models around 70-80 per each model and they were clean models from google, Pinterest other media sources as well photos made by myself for a bit more harsh environment worst resolution, light angle to achieve better recognition and performance of model.

Milestone 2: Labeling of Dataset (Completed: 20.10.2025)

After my collection of data set was done (although during the process some more image pop in into the dataset) I proceeded with labeling the pictures for model to be able recognizing the important part of each image each label contained correct model name also. So for the first time I tried to install labelling library to label each image internally in PyCharm itself although it resulted in many errors the UI was no clear the UI randomly closed during the work many times so after waiting some time I uninstalled and pip and I decided to try different approached I started using external site for labeling called Label studio and It was an excellent choice the website was clear smooth and although I had many picture to label the site itself does not interfere at all compared to previous on. It may seem like easy work and it was but due to many image the time I spent on labeling only I guess would be equal to 3-4 hours because the label should be as precise as possible and contain as small external sources than shoes to increase the model recognition ability I focus on edges so time consumed increased.

Label studio & Labeling example:



Figure 2. Labeling Nike Blazer Mid 77



Figure 3. Labeling Nike Dunk Low Purple Court

Milestone 3: CLIP integration (Completed: 27.10.2025)

Next point was to start experimenting with CLIP itself. I read plenty about it, so the implementation itself was not much problematic for me. Firstly, I set up the library open_clip library with the ViT-B-32 model wrote program to convert the test (labeled image) into vectors and to final obtain that visually similar shoes produced vectors that were mathematically close to each other

Script overview:

```
import json, faiss, torch, numpy as np
import open_clip
from PIL import Image
from pathlib import Path
from tqdm import tqdm

DEVICE = 'cpu' # keep CPU for your setup
MODEL_NAME, PRETRAINED = 'ViT-B-32', 'openai'
GALLERY = Path('catalog/images')
META_PATH = Path('catalog/metadata.json')
OUT_DIR = Path('index/vs')
OUT_DIR.mkdir(parents=True, exist_ok=True)

assert META_PATH.exists(), 'catalog/metadata.json not found. Create it first.'

print('[index] loading CLIP...')
model, _, preprocess = open_clip.create_model_and_transforms(MODEL_NAME, pretrained=PRETRAINED, device=DEVICE)
model.eval()

meta = json.loads(META_PATH.read_text())
vecs, kept = [], []
with torch.no_grad():
    for m in tqdm(meta, desc='Embedding gallery'):
        p = Path(m['image_path'])
        if not p.exists(): continue
        img = preprocess(Image.open(p).convert('RGB')).unsqueeze(0)
        z = model.encode_image(img)
        z = z / z.norm(dim=-1, keepdim=True)
        vecs.append(z.cpu().numpy())
        kept.append(m)

assert vecs, "No gallery images embedded."
X = np.vstack(vecs).astype('float32') # [N, D]
index = faiss.IndexFlatIP(X.shape[1]) # cosine via normalized vectors
index.add(X)
faiss.write_index(index, str(OUT_DIR/'index.faiss'))
```

Figure 4. Python Code for CLIP Prototyping

As a Summary for this point, I achieved the visually similar shoes produced vectors that were mathematically close (high cosine similarity), while distinct models were far apart which was helpful in next step when identifying the model based on this vector will come to the play.

Milestone 4: YOLO implementation (Completed: 30.10.2025)

In the next step I implemented YOLOv8n (n – Nano model) which is pre-trained Convolution Neural Network (CNN), of course it was a choice to reduce the time spent on model detection from my research it takes much time to train own CNN for designed task even so I do not possess knowledge for this operation so instead I am using readymade model although it can detect basic shapes and objects like: human, chair, cup only so I still need to expand the model by adding the shoe model and label into the model and exclude the rest of shape it detect like human, chair etc. I mentioned earlier to reduce chaos and made application only for Shoe detection.

From available models I had YOLO Nano, Small, Medium, v5, v8, v10. In the end I chose as mentioned earlier YOLOv8n for now my database is small, so my priority was to minimize latency, ensuring the system functions smoothly on standard hardware without requiring a dedicated heavy GPU so simply optimize and chose optimized parameters not to overkill or under chose the model parameters.

The main role of the YOLO model is to locate the sneaker within the camera frame. For that purpose, I have used the ultralytics library with an image size of 640x640. So, I created the .yaml file for the model to be able to recognize which object should be detected, what category, and where to place it.

```
path: ./data
train: images/train
val: images/val
names:
  0: adidas_ozelia_cream_blue
  1: new_balance_550_grey
  2: nike_blazer_mid77_darkblue
  3: nike_dunk_low_purple_court
  4: nike_pogo_plus_japanese_roots
```

Figure 5. Object Matching Criteria

Then when it was ready I finally started to compute and train model using CPU as a computation source (optionally could be done by GPU) using following parameters and commands:

- yolo detect predict model="Runs\detect\train2\weights\best.pt" source="data/images/val" save=True conf=0.25
- yolo detect train model=yolov8n.pt data="yolo_data.yaml" imgsz=640 epochs=30 batch=8 device=cpu workers=0 seed=42
- yolo detect val model="Runs\detect\train2\weights\best.pt" data="yolo_data.yaml" imgsz=640
- yolo detect predict model="Runs\detect\train2\weights\best.pt" source="data/images/val" save=True conf=0.25

I started the practice session with following parameters:

Epochs: 30 Batch Size: 8 Device: CPU optimizer: SGD (Stochastic Gradient Descent)

What does it mean to a non-programmers the epochs indicate full study cycle so 30 times of study times , batch size means how many picture AI groups questions then connect them into small packets of 8. Checks if it got them right, learns from its mistakes, and then moves to the next 8. CPU is a computation unit and optimizer to indicate the mathematic method AI uses to improve itself.

Results:

```
Image sizes 640 train, 640 val
Using 0 dataloader workers
Logging results to C:\Users\pabli\PycharmProjects\ComputerVisionPwr\Runs\detect\train2
Starting training for 30 epochs...
```

Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances	Size	
1/30	0G	0.8584	3.345	1.256	27	640: 100%	26/26 0.51t/s 57.0s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 3/3 0.71t/s 4.3s
	all	36	53	0.00523	0.907	0.183	0.143
Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances	Size	
2/30	0G	0.8094	2.725	1.232	26	640: 100%	26/26 0.51t/s 55.3s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 3/3 0.91t/s 3.4s
	all	36	53	0.502	0.148	0.24	0.158
Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances	Size	
3/30	0G	0.8557	2.451	1.254	22	640: 100%	26/26 0.51t/s 55.6s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 3/3 0.81t/s 3.8s
	all	36	53	0.382	0.252	0.263	0.187
Epoch	GPU_mem	box_loss	cls_loss	df_l_loss	Instances	Size	
4/30	0G	0.8683	2.32	1.248	25	640: 100%	26/26 0.51t/s 56.3s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100% 3/3 0.91t/s 3.4s
	all	36	53	0.154	0.449	0.218	0.151

Figure 6. YOLOv8n Training Process

```
30 epochs completed in 0.461 hours.
Optimizer stripped from C:\Users\pabli\PycharmProjects\ComputerVisionPwr\Runs\detect\train2\weights\last.pt, 6.2MB
Optimizer stripped from C:\Users\pabli\PycharmProjects\ComputerVisionPwr\Runs\detect\train2\weights\best.pt, 6.2MB

Validating C:\Users\pabli\PycharmProjects\ComputerVisionPwr\Runs\detect\train2\weights\best.pt...
Ultralytics 8.3.204 Python-3.11.9 torch-2.8.0+cpu CPU (AMD Ryzen 5 5600H with Radeon Graphics)
Model summary (fused): 72 layers, 3,006,623 parameters, 0 gradients, 8.1 GFLOPs
```

	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
	all	36	53	0.776	0.833	0.885	0.766
adidas_ozelia_cream_blue		8	10	0.637	0.9	0.914	0.8
new_balance_550_grey		9	13	0.791	0.692	0.849	0.752
nike_blazer_mid77_darkblue		8	15	0.761	0.85	0.78	0.606
nike_dunk_low_purple_court		4	6	0.97	0.833	0.942	0.832
nike_pogo_plus_japanese_roots		7	9	0.718	0.889	0.939	0.842

Speed: 1.5ms preprocess, 65.2ms inference, 0.0ms loss, 6.2ms postprocess per image

Figure 7. YOLOv8n Training Process Summary

In the final training, it was successful completed with the following nerd data like image number, percentage of label recognition, region of interest, and based on this data I generated the plot data to visualize it.

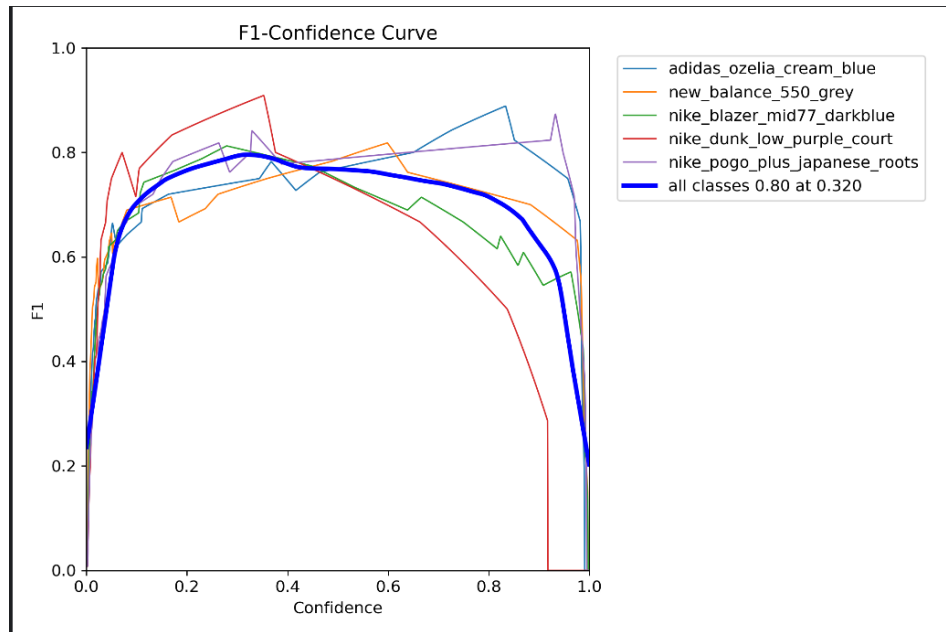


Figure 8. YOLOv8n Summary Plot

From the following curve we can conclude some useful data like a peak of F1 score equal to 0.8 for 0.32 confidence score. Which tells us that model training is reliable, achieving this 0.8 score for 0.32 confidence threshold. For some comparison the best performer was `nike_dunk_low_purple_court` and the worst was `new balance competitor` and for other project comparison it is also standard score I found there are messy project that achieves 0.4 – 0.6 accuracy score, but saw some with 0.95 + score which guarantees the best performance I am in the middle which should be taken into account for final result presentation.

At the end of YOLO phase (post-training) I added additional python code:

```
import os, cv2, json
from pathlib import Path
from ultralytics import YOLO

SRC = "data/images/val"
OUT = "crops/val"
MODEL = r"Runs/detect/train2/weights/best.pt"
CONF = 0.25

Path(OUT).mkdir(parents=True, exist_ok=True)
model = YOLO(MODEL)

for img_path in Path(SRC).glob("*."):
    if img_path.suffix.lower() not in [".jpg", ".jpeg", ".png", ".bmp", ".webp"]:
        continue
    res = model.predict(source=str(img_path), conf=CONF, verbose=False)
    if not res: continue
    im = cv2.imread(str(img_path))
    for i, b in enumerate(res[0].boxes):
        x1,y1,x2,y2 = map(int, b.xyxy[0].tolist())
        # optional padding 5%
        h, w = im.shape[:2]
        pad_x = int(0.05*(x2-x1)); pad_y = int(0.05*(y2-y1))
        x1p = max(0, x1-pad_x); y1p = max(0, y1-pad_y)
        x2p = min(w-1, x2+pad_x); y2p = min(h-1, y2+pad_y)
        crop = im[y1p:y2p, x1p:x2p]
        out_name = f'{img_path.stem}_{i}.jpg'
        cv2.imwrite(str(Path(OUT)/out_name), crop)
```

Figure 9. Cropping From Predictions

That has a quality of life purpose to extract the region of interest (ROI) defined by the bounding(label) box and add a 5% additional padding to ensure the entire shoe is captured before passing it to the classifier in case not to miss some crucial detail on image that was identified by model before and ensure reliable result.

Milestone 5: Adding the FAISS (Completed: 07.11.2025)

The next phase of the project is getting familiar with FAISS (Facebook AI Similarity Search) explained at the beginning. It works really well paired with the CLIP technique as a reminder it convert the test image into vectors and to final obtain that visually similar shoes produced vectors that were mathematically close to each other the FAISS enhance this procedure by compares the live vector against a pre-indexed database in meantime. I divided FAISS into two phases 1. Indexing (Database Creation) and 2. Querying (Real-time Search)

For this I used following code:

```
import json, faiss, torch, numpy as np
import open_clip
from PIL import Image
from pathlib import Path
from tqdm import tqdm

DEVICE = "cpu" # keep GPU for your setup
MODEL_NAME, PRETRAINED = "vit-B-32", "openai"
GALLERY = Path("catalog/images")
META_PATH = Path("catalog/metadata.json")
OUT_DIR = Path("indexes/v1")
OUT_DIR.mkdir(parents=True, exist_ok=True)

assert META_PATH.exists(), "catalog/metadata.json not found. Create it first."

print("[index] loading CLIP...")
model, _, preprocess = open_clip.create_model_and_transforms(MODEL_NAME, pretrained=PRETRAINED, device=DEVICE)
model.eval()

meta = json.loads(META_PATH.read_text())
vecs, kept = [], []
with torch.no_grad():
    for m in tqdm(meta, desc="Embedding gallery"):
        p = Path(GALLERY / m["image_path"])
        if not p.exists(): continue
        img = preprocess(Image.open(p).convert("RGB")).unsqueeze(0)
        z = model.encode_image(img)
        z = z / z.norm(dim=-1, keepdim=True)
        vecs.append(z.cpu().numpy())
        kept.append(m)

assert vecs, "No gallery images embedded"
x = np.vstack(vecs).astype("float32")

index = faiss.IndexFlatIP(x.shape[1])
index.add(x)
faiss.write_index(index, str(OUT_DIR / "index.faiss"))
np.save(OUT_DIR / "vectors.npy", x)
(OUT_DIR / "meta.json").write_text(json.dumps(kept, indent=2))
print(f"[index] saved {len(kept)} items to {OUT_DIR / 'index.faiss'}")
```

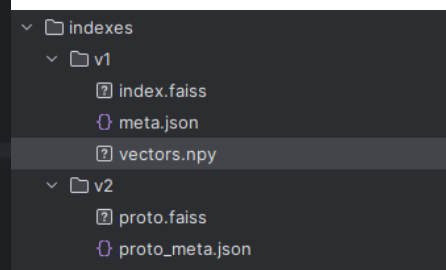


Figure 9/10. FAISS Indexing and appropriate files

To be specific what this code do To ensure high accuracy and robustness, the indexing process implements a "Class Prototype" strategy which divide into:

1. Vectorization: Every image in the training set is converted into a high-dimensional vector using the CLIP model.
2. Normalization: All vectors are L2-normalized. This ensures that the dot product calculation performed by FAISS is mathematically equivalent to Cosine Similarity, which is the optimal metric for comparing semantic embeddings.
3. Prototype Averaging: Instead of storing every single noisy training image, the system calculates the mathematical mean (average) of all vectors belonging to the same shoe model.

Benefit: This creates a single, stable "Prototype Vector" for each shoe model, significantly improving robustness against lighting variations and bad camera angles.

But for simplicity we can conclude it to such statement: This code allows FAISS to iterate through the gallery images, generate CLIP embeddings, and store them in a .faiss index file

Phase 2 for FAISS - Querying

```
import json, faiss, torch, numpy as np
import open_clip
from PIL import Image
from pathlib import Path

DEVICE = "cpu"
MODEL_NAME, PRETRAINED = "ViT-B-32", "openai"
INDEX = "indexes/v1/index.faiss"
META = "indexes/v1/meta.json"

model, _, preprocess = open_clip.create_model_and_transforms(MODEL_NAME, pretrained=PRETRAINED, device=DEVICE)
model.eval()
index = faiss.read_index(INDEX)
meta = json.loads(Path(META).read_text())

def embed(p): 1 usage
    with torch.no_grad():
        img = preprocess(Image.open(p).convert("RGB")).unsqueeze(0)
        z = model.encode_image(img)
        z = z / z.norm(dim=-1, keepdim=True)
        return z.cpu().numpy().astype("float32")

def search(image_path, k=5): 4 usages (3 dynamic)
    q = embed(image_path)
    sims, ids = index.search(q, k)
    out=[]
    for s, i in zip(sims[0], ids[0]):
        m = meta[int(i)]
        out.append((float(s), m["class_name"], m.get("model_name", ""), m["image_path"]))
    return out

if __name__ == "__main__":
    # pick one of your crops
    crop = sorted(Path("crops/val").glob("*.jpg"))[0]
    print(f"Query: {crop}")
    for s, cls, name, path in search(str(crop), k=5):
        print(f'{s:.3f} {cls:40s} {name} <- {path}')
```

Figure 11. FAISS Querying Code

Here it comes also technical detail of the code itself:

1. **Input:** The system receives the cropped image of the shoe from the YOLO model detection.
2. **Embedding:** The live image is passed through the same CLIP model to generate a query vector.
3. **Search:** FAISS compares this query vector against the prototype vectors stored in the index.
4. **Ranking:** The system returns the Top-K nearest neighbors (closest matches). The result with the highest similarity score (closest to 1.0) in the final, this is the final search resulting from the selected shoe model that was identified.

Also, the shortcut version here: During this stage FAISS in real-time when the camera is running takes the detected shoe if was firstly identified by YOLO detector and finds its identity in the database with increased accuracy connection CLIP & FAISS vector search.

That would be all from the technical backend side of the application, the operation of the system and core 'intelligence' and its operation of work. By combining YOLOv8 for precise object localization, CLIP for semantic feature extraction, and FAISS for high-performance similarity search, the backend is capable of identifying specific sneaker models in real-time.

Now comes less technical segment that focuses on the application layer specifically, so simply Graphical User Interface (UI) and still in progress market data showcase through the Price API Implementation.

Milestone 6: User Interface (UI) (Completed: 10.11.2025)

Next comes probably the easiest and most relaxed part of a project for me and probably many other students that decided on implementing user interfaces into their projects. When it comes to my project nothing exaggerated for now

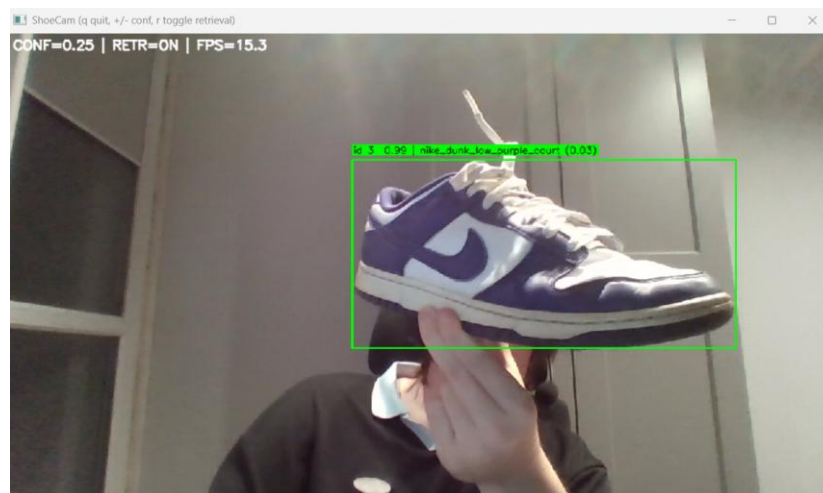


Figure 12. User Interface with parameters

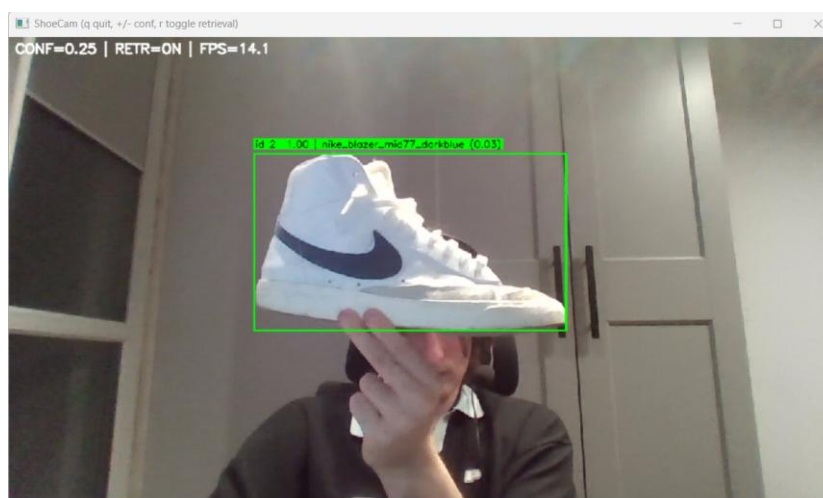


Figure 13. User Interface with parameters

So, in the UI it is shown a dynamical frame that tracks the shoe perimeter, and I must say I am very satisfied with this specific result. Meaning the detector is very accurate in when it comes to tracing always the shape of shoe is highlighted and only them even when there are multiple items and shape in the camera those will not detected as shoe even when they could have similar shape indicating my good job when it comes to labeling and model training in shoe detection overall .



Figure 14. UI for No-Shoe appearance

When it comes to describing other parameters in the left high corner we have **CONF** variable which indicates the confidence of the detection by default 0.25 but with plus (+) or minus (-) sign can be increased or decreased depending on the situation for too high the system miss shoes in bad lighting. And if it is too low for causes, it to mistake random objects like in the picture above, so I find the value 0.25 to be the optimal starting point.

Then **RETR** the short cut for retrieval this is a status indicator that tells you whether the search/march mechanism (FAISS) is currently active ON or OFF

FPS so frame per second my laptop does not have too quality camera the video oscillate between 10-20 FPS so it is not high which may highly influence the result of the detection compared to more precise cameras with higher FPS values, although I was prepared for harsh environment and prepared such photos for model to detect it. Although I am curious how the YOLO + CLIP + FAISS operates in a more quality environment and if the result would be reliable and fastest.

We can also observe when Sneaker is present the **ID** so number that is associated with every shoe model that was though simultaneously with YOLO,CLIP,FAISS it indicates also for FAISS where to look and output in dataset and Number indicating the Area of Shoe present on camera if 100% is shown then result with 1.00 and everything below >1 with approximately the value model 'think' is present according to training we did and its own 'speculations'.

Lastly the color of the Border when Shoe is detected so :

- ☒ Green Box tells us High Certainty (>55%).
- ☐ Gray Box & "...": Unstable. The shoe is moving too fast or blurring, so no ID is attempted.

Milestone 7: Pricing & Web Scrapping

I hesitated for some time to put my efforts into this part of the project to start developing. Mainly because I was a bit anxious web scrapping is a bit hard and may have some difficulties implemented, although that all was fake feeling at first because it was not that hard second, I failed miserably.

To start with I made some research about **DuckDuckGo Search API** and get familiar with principle of operation coding it in python and I thought what could be wrong I started testing it with quotes like nike, adidas [model name] price stockx/goat etc. however any attempt was not successful

```
results = DDGS().text(search_query); max_results=2)
[Scraper] Searching for: Nike Dunk Low Purple Court price stockx goat...
Result: Price Not Found

[Scraper] Looking for: Nike Dunk Low Purple Court...
C:\Users\pabli\PycharmProjects\ComputerVisionPwr\scripts\price_scraper.py:23: RuntimeWarning:
  results = DDGS().text(q, max_results=2)
C:\Users\pabli\PycharmProjects\ComputerVisionPwr\scripts\price_scraper.py:23: RuntimeWarning:
  results = DDGS().text(q, max_results=2)
[Scraper] ✗ No results for 'Nike Dunk Low Purple Court price goat sneaker'
[Scraper] Price not found in search snippets.
Final Result: N/A
```

Figure 15. Web Scrapping Results

So, I tried to find where the problem lies and found out it probably was captcha/bots firewall to be unable to get the proper results from sites I was looking at. The main issue lies I had to use DuckDuckGo because it is free to use system and could not afford any other alternative for most reliable ones Stockx, GOAT any other I would need permission, business cooperation etc. that is not in the range for me. I also tried to find any other free API and encounter **KickDB** although for free subscription I will have 1,000 requests per month on Standard API so with some AI chat we got to conclusion with our current FPS, camera stability, calibration the I would access 1,000 requests with 33 seconds of running loop so I would be able with best scrap 1-2 shoes prices per month for free so I guess this was not a solution also, I tried to find other alternatives although vainly.

So, as the back-up plan, I implemented the Mock Database for current Models to mimic the web scrapping behavior. It was the easiest way to do just brief research on market prices and hardcoding the price range into the program, and although it works well, I feel disappointed. I really tried to implement live web scrapping into the program but in the end, this is not achievable without additional permission.

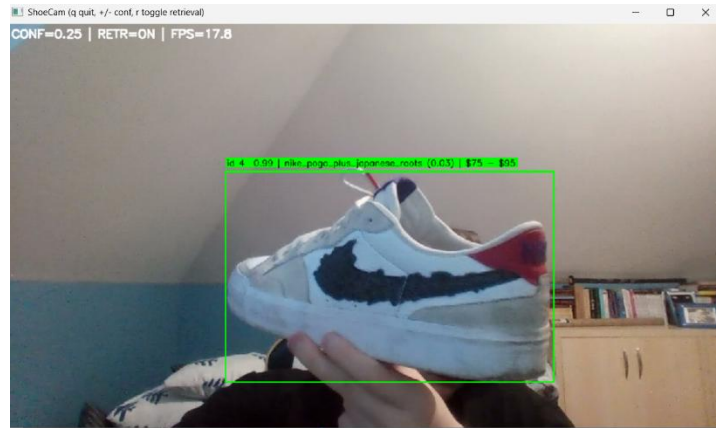


Figure 16. Detected Snicker with Price from Mock Database

6.Chellanges Encountered During the Project

The sufficient results are one and the path to achieve them is second. I would like to stop a bit and discuss them briefly for further understanding of the reader as well as for my own consideration.

6.1. Data Gathering & Labeling Constraints

As I mentioned earlier seems easy but there is hidden issue related with photo gathering firstly I gathered around 300 for 5 models so 70-80 per model which is actually bare minimum from what I have read the optimal image amount should oscillate around >150 although doing this would result with even better results. These 300 photos I collected give satisfying result but even when I would like to match such standard 150+ per model I would have trouble because the database Pinterest google shops have limited amount of photo so for some model gathering those 80 images was a problem. Then for Labeling also for 300 pictures the amount of time I spent with sufficient precision at the end bloom the excellent result but when I would add another 300 images the time spent on project would increase not that I did not enjoy it , but rather those 3-4 hours could be invested in more crucial part or new features rather than pure labeling case.

6.2. YOLO Model Training Interruptions

First, I would like to emphasis this may not be the challenge rather than slight difficulty. I mean the training with 30 epochs and CPU as main computation unit took me approximately around 2 hours so during this time I had to pause the work and do something else so It was not a challenge but unmotivated when I was warmed up wanted to procced improve and had to stop for 2 hours (BTW due to this I made pause for 2 days)

6.3. Real-time Latency Optimization

Balancing the heavy computation of CLIP and FAISS with the frame rate of the webcam required some optimization processes I made on my way to this. Also, some hours spent just to optimize the camera view and at the end the results are not superior. This for sure would be easier when working with better equipment, which unfortunately was not available by the time of developing the application.

6.4. Visual Similarities

Distinguishing between very similar models Dunk Low vs. Japanese Pogo posed a challenge for the classification model. As we can observe on Figure 8, some models had higher and some lower confidence scores so more reliable results and that can be seen in results some models are faster to detect than others. However, to that add problem like visual similarities which happened between the models mentioned above, resulting in mismatch of the name and id due to similarities, small database, model inaccuracies, low FPS resolution.

6.5. API Restrictions

The problems with real-time market data were maybe not one of the biggest but troublesome challenges. Using the DuckDuckGo Search API, I first tried to create a web scraper to retrieve real-time prices from marketplaces like StockX and GOAT. But problems appeared when automated search queries were routinely denied by the stringent anti-bot procedures used by major e-commerce platforms like Cloudflare or CAPTCHA, leading to connection timeouts. The Alternative KicksDB was firstly a valid substitute; I investigated utilizing the KicksDB API. Nevertheless, there is a monthly cap of 1,000 requests for the free tier. A real-time solution would use the monthly allowance in less than one minute of runtime because the computer vision pipeline operates at about 15–30 frames per second.

Before I jump up into the conclusion, I will quickly show the project tree and how it extended across the work development so on the beginning there was only few files however now I have them so many I cannot fit them in one screen:

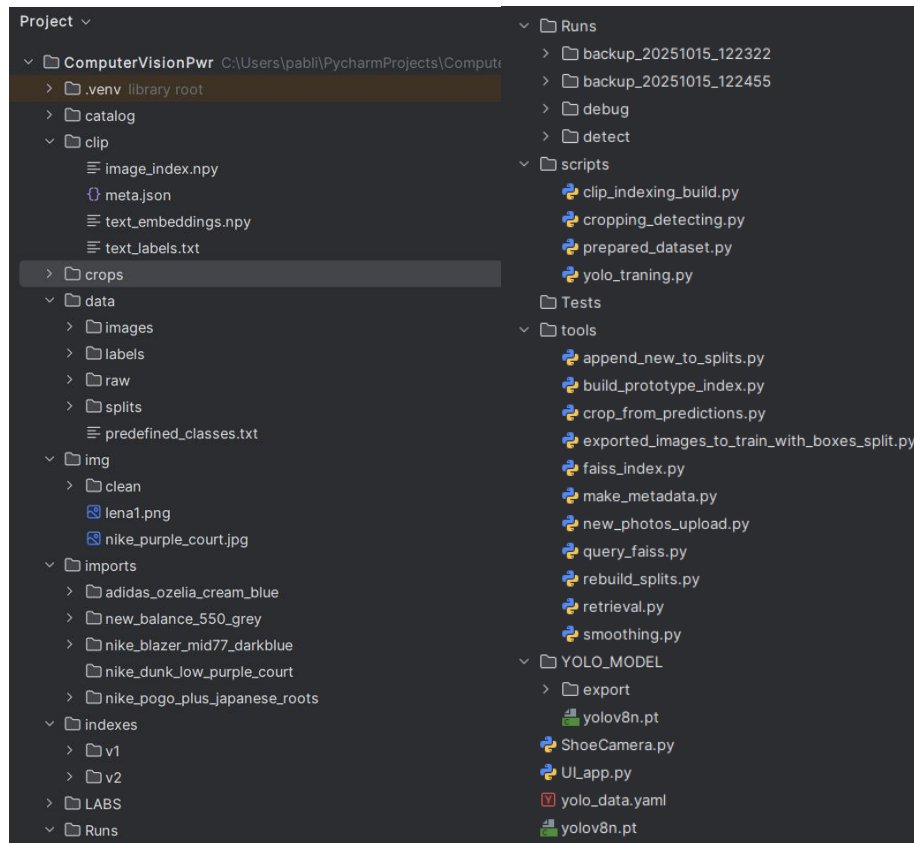
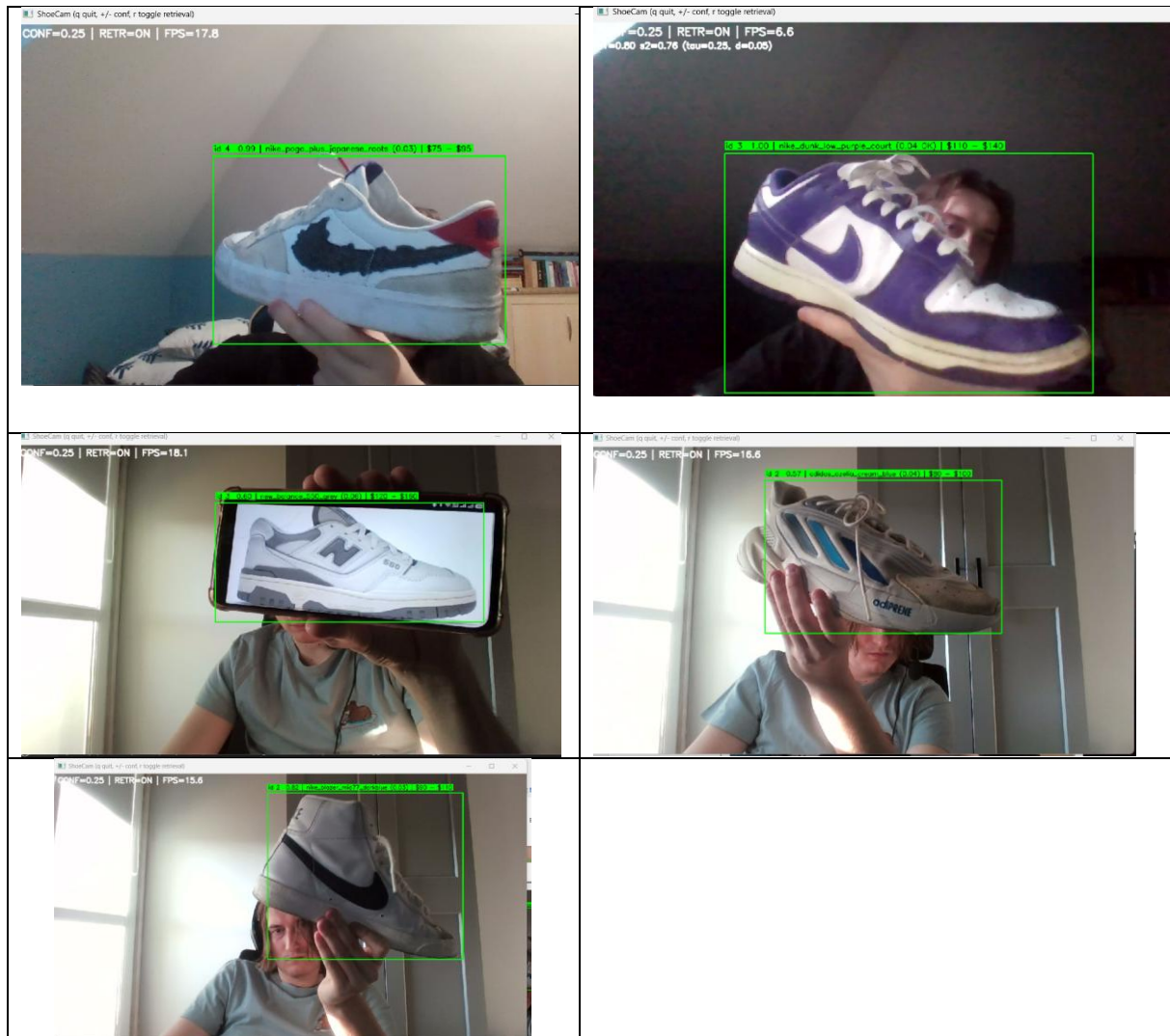


Figure 17. Project File Structure

7. Conclusions

To sum up everything in the final, I have achieved my three goals stated before actual work started meaning Real-time Detection, Model Classification, Accuracy of the output Data. The final application is able to detect in real time with reliably model classification without many errors on the way, although the speed of recognition is not too fast and resolution FPS of the view is not too much the program does its job which hugely satisfy me. There were for sure times where everything went great and fast and periods when obstacles and errors happened, increasing the difficulty of achieving desired results. I have learnt a lot about real time detection, AI, similarity searching algorithms, and training your own model for desired tasks. I feel the project for sure was fun and enjoyable but also there is a lot ground to improve, optimize , refine change and I do not have as much time to spent on that due to other project and not sufficient amount of time expand database for more options in the future.

Here are some other shoe detection final results:



8. Links

GitHub repository: https://github.com/cwik3/Shoe_model_Detector